Assignment: Assignment 5 Strategy Pattern

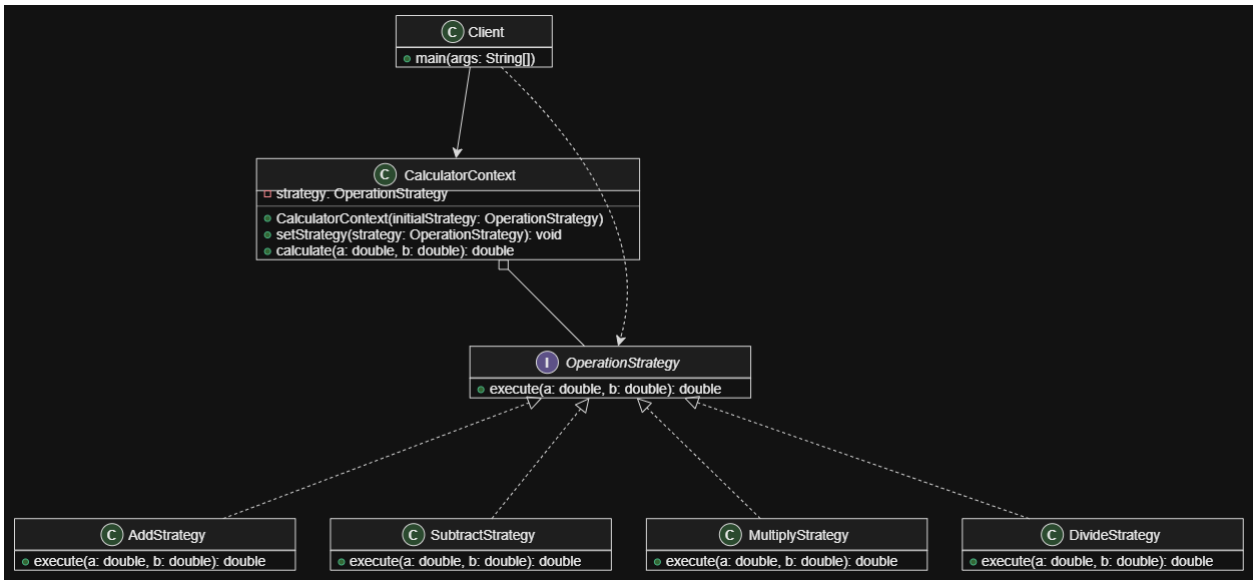
Agabekov Akzhigit SE2523

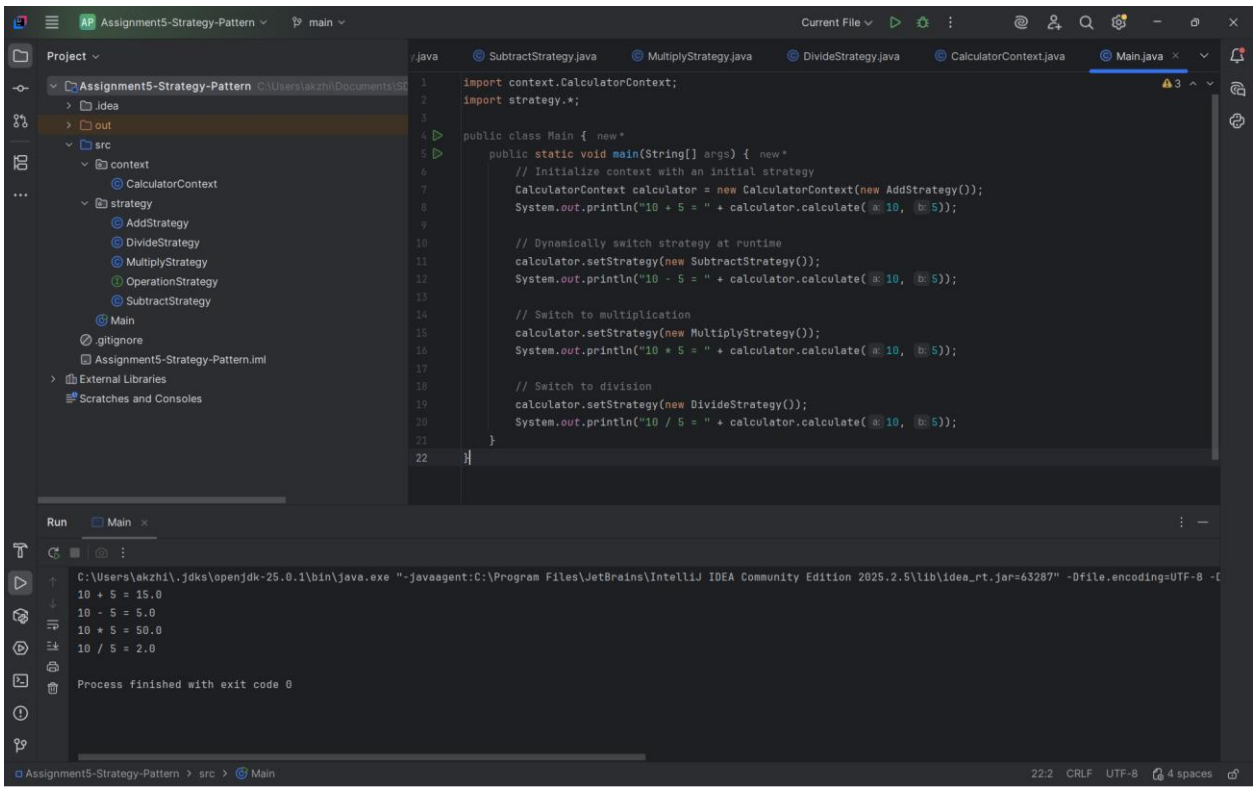
1. Introduction

The objective of this assignment is to implement the Strategy Behavioral Design Pattern in Java. In software systems, mathematical processing or algorithm selection often relies on complex conditional blocks (if/else or switch/case).

Using the Strategy pattern, algorithms are encapsulated into separate interchangeable classes conforming to a common interface (OperationStrategy). The CalculatorContext delegates the execution to the current strategy without knowing its concrete implementation details.

2. Structural Architecture & UML Diagram





3. Clean Code Principles Applied

1. **Elimination of Conditional Logic via Polymorphism (Clean Code Ch. 10):**
    - Instead of using switch (operationType) inside the calculator, the operation is delegated directly to strategy.execute(a, b).
2. **Meaningful, Intention-Revealing Names:**
    - Concrete strategies (AddStrategy, SubtractStrategy, DivideStrategy) explicitly state their algorithm responsibility.
3. **Single Responsibility Principle (SRP):**
    - Each concrete strategy class encapsulates exactly one mathematical algorithm.
4. **Context Algorithm-Agnosticism:**
    - CalculatorContext holds a reference to the abstract OperationStrategy interface and remains completely unaware of concrete class internals.
5. **Open/Closed Principle & Easy Extensibility:**

Adding new operations (e.g., ModulusStrategy or PowerStrategy) requires creating a new class implementing OperationStrategy without modifying CalculatorContext or existing strategy classes.

4. Strategy vs. State Pattern Reflection

While **Strategy** and **State** patterns share a similar structural class diagram (both wrap an interface reference and delegate work), their **intent and state-transition management differ fundamentally**:

Strategy Pattern: The client explicitly configures and passes the appropriate strategy to the context. Strategies are interchangeable algorithms that are generally independent of one another and do not trigger transitions between themselves.

State Pattern: The context's behavior changes dynamically based on its internal state. Concrete states often hold references to the context and trigger transitions to other states automatically during execution.

5. Conclusion

Flexibility Gained: The system can dynamically change execution behavior at runtime without modifying the context class or using nested conditional branches.

Classes Added: Extra strategy classes are introduced, but this trade-off significantly improves readability, testability, and adherence to the Open/Closed Principle.

Github: https://github.com/Akzhig1t01/Assignment5-Strategy-Pattern